An Introduction to Perceptrons and their Viability in Branch Prediction

Thomas Bioren, Duy Le

***Abstract*—This paper answers the following question: "Are perceptrons a viable approach to the problem of accurate branch prediction?" In the rapidly evolving landscape of superscalar computer architecture, branch prediction continues to prove instrumental. Fast and accurate branch prediction is paramount in improving processor performance and reducing pipeline stalls. This paper introduces pipelining, branch prediction, and a novel approach to prediction: neural networks (although we focus on one specific neural network: perceptrons). We begin by introducing the reader to the concept of instruction level parallelism, pipelining, and issues it brings. We dive into branch prediction and how it aims to solve this issue. We introduce the concept of perceptrons and how they work. Finally, we show how perceptrons are currently being used in branch prediction and their performance relative to other approaches. This report highlights the immense potential machine learning algorithms have in the space of branch prediction.**

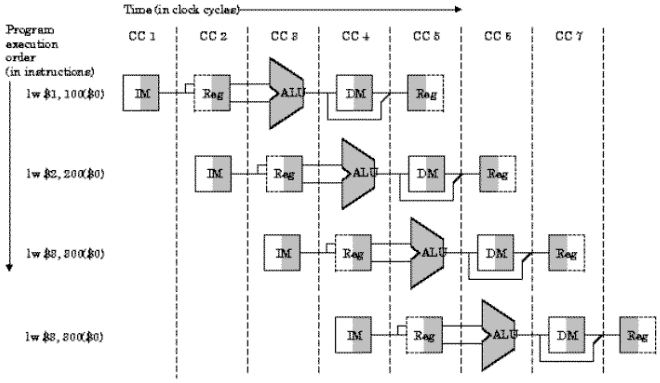

Fig. 1. A pipeline diagram of a 5-stage pipelined processor.

I. An Introduction to Branch Predictors and Perceptrons

To fully understand the concept of perceptron-based branch prediction, a brief overview of both perceptrons and branch prediction is provided for the reader.

A. What Is a Branch Predictor?

The aim of a branch predictor is to minimize the number of pipeline flushes in a processor. Modern processors employ a technique called *pipelining* to implement instruction-level parallelism (ILP). A pipelined processor achieves ILP by splitting instructions into pipeline stages (usually, the stages are instruction fetch, decode, execute, memory, and writeback). As Figure 1 shows, with split up instructions, the processor can execute one instruction's fetch stage, another instruction's decode stage, and so on—all in one cycle. This is extremely powerful. As an example, if Figure 1's 5-stage pipelined processor were simply a 5-stage non-pipelined multicycle processor, it would execute one instruction every 5 cycles (one cycle per processor stage). With a full pipeline, the processor executes one instruction every cycle. This is a 500% improvement in cycles per instruction (CPI).

However, pipelining has one fatal flaw: it begins executing instructions without knowing the results of previous instructions. In most reduced instruction set (RISC) processors, the processor cannot know whether to take a branch until the branch instruction reaches the execute stage of the pipeline. This means that multiple instructions *after* the branch instruction are loaded into the pipeline before the processor knows what to execute. If the branch instruction is evaluated and the processor does not take the branch, everything is fine and the loaded instructions are accurate. However, if the processor does need to take the branch, the instructions loaded into the pipeline are incorrect and the processor needs to get rid of them. It does this by flushing the pipeline—inserting no operation (NOP) instructions into the pipeline and re-loading the correct instructions. This is extremely detrimental to CPI because the pipeline is no longer full. The CPI now falls from what would be 1 to maybe 2 or 3. As you can see, being able to properly predict which instructions go next in a pipeline is a valuable trait in a processor. This is where branch predictors come in.

Branch predictors try to guess whether a branch will be taken before the next instructions are loaded into the pipeline. They generally use clues from previous branches and the state of the program to predict which instructions the processor should load next.

B. What Is a Perceptron?

In short, a perceptron is a single-layer neural network [1]. It is designed to classify linearly separable data. These classifications can be anything. In our case, they are "take" and "not take." Figure 2 shows an example of linearly separable data, non-linearly separable data, and inseparable data. As you can see, linearly separable data is simply data that can be separated by drawing a line between its two categories. It is important to note that, although the second dataset in Figure 2 is not linearly separable in cartesian coordinates, it is linearly separable in polar coordinates [4].

You can think of the blue crosses as "take" and the black squares as "not take." A perceptron is designed to learn where to draw a line to separate the data. If a datapoint falls above

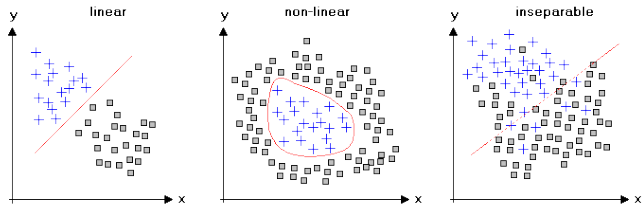

Fig. 2. Linearly separable data, non-linearly separable data, and inseparable data, all in cartesian coordinates.

this line, the perceptron will tell the processor to take the branch, if it falls below, the perceptron will tell the processor to not take the branch. An important note is that this math holds for any $n > 1$ dimensions. This description will stick to 2 dimensions because it is easier to visualize.

C. How a Perceptron Makes Predictions

The math behind a perceptron is relatively simple. There are three parameters: ω—the coefficient vector, *r*—the learning rate, and *x*—the input vector [2]. For our dataset above, the parameters are shown in Figure 3.

$$\omega = \begin{pmatrix} \omega_0 \\ \omega_1 \\ \omega_2 \end{pmatrix}$$

$$r = \textit{learning rate}$$

$$x = \begin{pmatrix} 1 \\ x \\ y \end{pmatrix}$$

Fig. 3. The definitions of the parameters in a perceptron.

The coefficients vector, ω, is always one more dimension than the number of dimensions of our dataset and represents the line that dissects the dataset in two. In this case, ω has three dimensions because our dataset is two-dimensional. The learning rate, r, is some number, usually between 0 and 1 that defines how willing our perceptron is to change the line it is guessing at. A higher number means the line will change faster but has a higher chance of overcorrecting. Finally, the input vector, x, represents the coordinates of our dataset. It always has 1 more dimension than the number of dimensions of the dataset. This extra dimension always has the value 1 as the constant term.

To classify a datapoint, we simply take a dot product. In this example, we are trying to classify a point $p = (1, 3)$ in our dataset with the values in Figure 4 for our perceptron. As Figure 5 shows, we need to use a dot product to find the classification for our point p. If $\omega \cdot x > 0$, the datapoint is classified as "take." Otherwise, it is classified as "not take." Because $12 > 0$, the perceptron branch predictor will tell the processor to take the branch. You can see this data lines up with the data provided in the linearly separable data in Figure 2.

$$p = (1, 3)$$

$$\omega = \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}$$

$$r = 0.2$$

$$x = \begin{pmatrix} 1 \\ x \\ y \end{pmatrix}$$

Fig. 4. Example values for a perceptron

$$\omega \cdot x = 1 * 1 + 2 * 1 + 3 * 3 = 12$$

$$12 > 0 \rightarrow \text{"}take\text{"}$$

Fig. 5. An example classification of a dataponit with values from Figure 4.

D. How a Perceptron Learns

The perceptron learns by changing its coefficients vector, ω, to create a line that perfectly dissects the "take" and "not take" datapoints. It does this with the following steps. For each element in the dataset, $\omega_i' = \omega_i + r * d$, where ω_i' is the new ω_i, r is the learning rate, and d is if the should be above or below the line. D is 1 if the datapoint should be above the line and -1 if it should be below. As an example, assume that $p = (1, 3)$ from the previous example was actually supposed to be below the line and our perceptron erroneously placed it above. Figure 6 shows how we would update the coefficients vector.

$$\omega_0^{'} = \omega_0 + r * d * x_0 = 1 + 0.2 * (-1) * 1 = 0.8$$

$$\omega_1^{'} = \omega_1 + r * d * x_1 = 2 + 0.2 * (-1) * 1 = 1.8$$

$$\omega_2^{'} = \omega_2 + r * d * x_2 = 3 + 0.2 * (-1) * 3 = 2.4$$

$$\omega' = \begin{pmatrix} 0.8 \\ 1.8 \\ 2.4 \end{pmatrix}$$

Fig. 6. An example showing how to update the coefficients vector, ω, with the data from previous figures.

As Figure 7 shows, the perceptron still classifies point p as "take," but it is much closer to being "not take." If we had increased its learning rate, it may have gone all the way to "not take" in this one step. A large part of designing perceptrons is

fine-tuning the learning rate, similar to tuning a PID loop. This is important because it demonstrates the perceptron's ability to rapidly adjust to new patterns, a vital attribute of branch predictors.

$$\omega \cdot x = 0.8 * 1 + 1.8 * 1 + 2.4 * 3 = 9.8$$

$$9.8 > 0 \rightarrow \text{take}$$

Fig. 7. A re-run classification of point p with a new coefficients vector.

II. The Problem with Traditional Branch Predictors: History Length

Traditional branch predictors use pre-defined logic to predict branch behavior. For example, a GShare predictor is a global predictor with index sharing, which uses XOR for branch address and global history. However, traditional branch predictors, especially 2-level branch predictors, cannot properly accommodate long branch history length. Consequently, when the correlated branches have a longer distance between them than the length of the global history register, the misprediction rate can increase.

Although there are many ways to address this problem, such as using a Pattern History Table (PHT) or a more complex method like variable length path branch prediction, they all have significant disadvantages. For example, a PHT-based scheme requires more space as well as longer training times. The variable length path branch predictor has a complex and impractical architectural implementation [3].

III. How Perceptron-Based Branch Predictors Address the Problem

Because of the linear scalability of perceptron size and its inputs, the perceptron-based branch predictor allows linear scalability between hardware resources and history length. This lets it take advantage of the longer history length while maintaining reasonable hardware resources and training time. This helps improve the accuracy of prediction and, hence, improve overall performance.

In Figure 8, we can see that the accuracy of the perceptron predictor improves with history length, while GSHARE's accuracy bottoms out at 18.

For a fixed hardware budget, perceptron-based predictors can consider greater history length compared to traditional two-level adaptive schemes using PHTs. This is because the size of perceptrons can grow linearly with its inputs, which is the branch history. Meanwhile, the size of PHTs grow exponentially with history length since it is indexed by the branch history. [1]

Regarding training times, perceptron-based predictors can learn faster than traditional predictors using PHT indexing. For predictors using perceptrons, the training time is independent of the length of branch history. On the other hand, although PHTs allow traditional predictors to consider greater branch history length, their training time depends on the length of branch history [3].

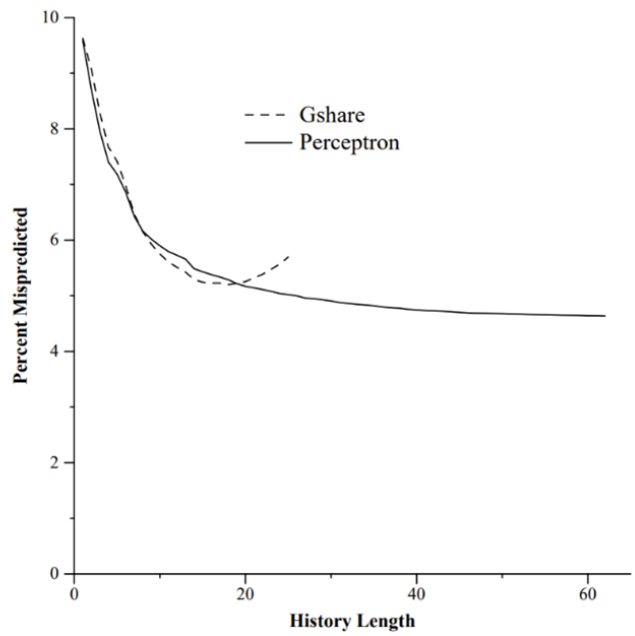

Fig. 8. History length vs. performance comparison between a GShare predictor and a perceptron predictor.

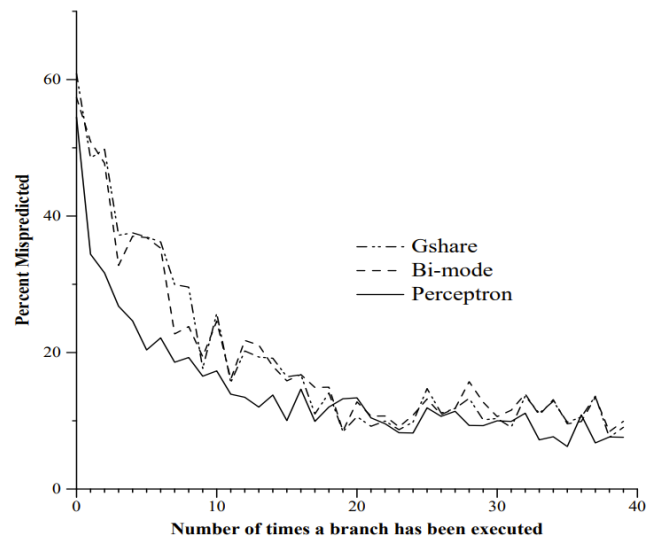

Fig. 9. Average training times for executing GCC. The x-axis is the number of times a branch has been executed. The y-axis is the average, over all branches in the program, of 1 if the branch was mispredicted, 0 otherwise.

The statistic in Figure 9 tracks how fast each predictor learns. We can see that the perceptron predictor achieves greater accuracy more quickly than the other two predictors.

Therefore, because of their ability to use longer branch history, perceptron-based branch predictor performance improves greatly. It is worth noting that, despite the increase in performance due to longer branch history, the perceptron predictor will work best when it is used to predict linearly separable behavior.

IV. Limitations of Perceptrons

Despite having advantages over traditional branch predictors, perceptron-based predictors still have limitations. Designed to classify linearly separable data, the potential of a perceptron might not be fully utilized when working with linearly inseparable data. Meanwhile, two-level PHT-based predictors such as GSHARE can work with both linearly separable data and linearly inseparable data as long as there is enough training time [3].

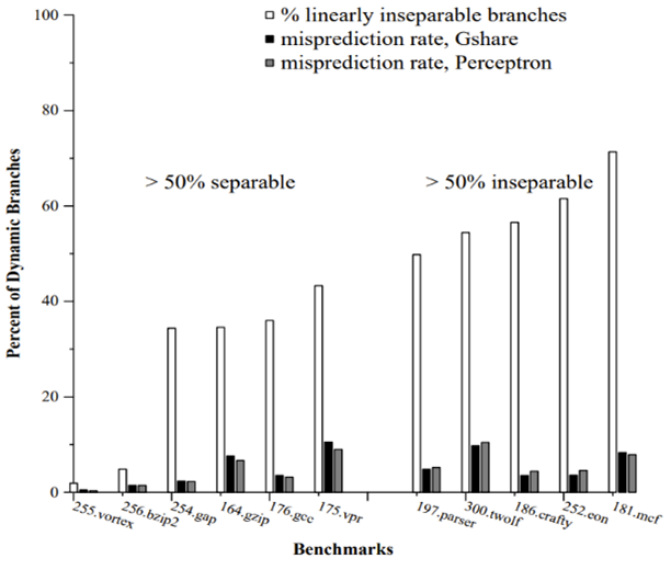

Fig. 10. Linear separability vs. Performance between GSHARE predictor and Perceptron predictor

Figure 10 shows that when the dynamic branches are mostly linearly separable, the perceptron predictor is better than GSHARE. Otherwise, GSHARE tends to be more accurate.

V. Perceptron Applications in Branch Prediction

Perceptron predictors can be some of the simplest applications of neural networks in branch prediction. This simple perceptron method can be applied to two kinds of branch prediction strategies: global/local perceptron prediction and overriding perceptron prediction.

VI. Global/Local Perceptron Prediction

Alloyed predictors—a hardware-based two-level branch predictor organization that combines global and local history in the same structure—can even outperform hybrid predictors that select from between a local and global predictor. When incorporated with perceptrons where some perceptrons get input from the global history register and others receive input from the local history register, the performance can be further improved.

The perceptron scheme and the alloyed scheme together increase the accuracy of the predictor. While the perceptron part makes it outperform the traditional two-level branch predictor, the combination of global and local history makes this perceptron predictor more accurate than a perceptron predictor using only a global or local history table.

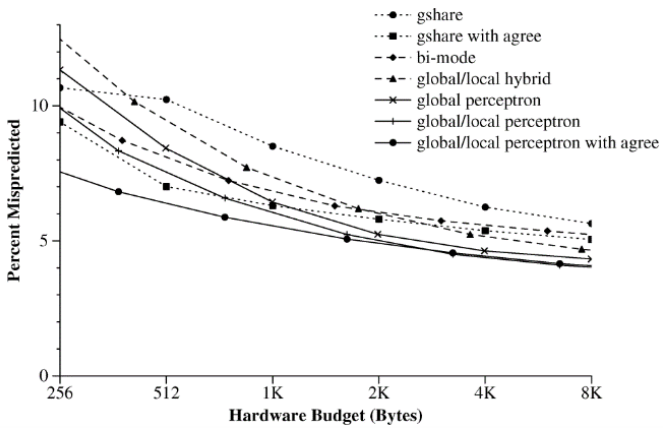

Fig. 11. Accuracy for perceptron predictors and traditional predictors for corresponding hardware budget

From Figure 11, we can see that the misprediction rate of a global/local perceptron predictor with an agree mechanism is consistently the lowest throughout the hardware budget, which indicates the accuracy of the perceptron predictors is superior in most cases, especially with a larger budget.

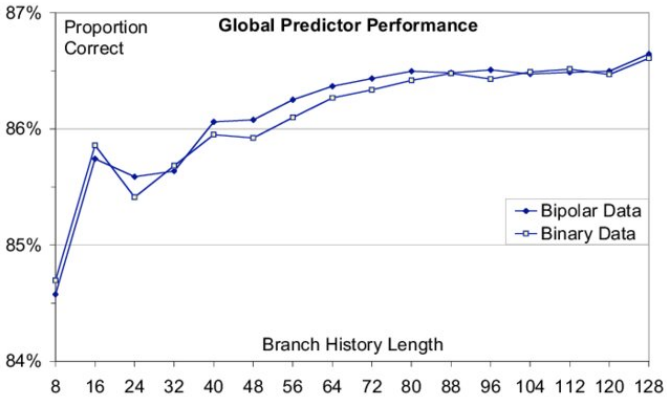

Fig. 12. Accuracy of global only perceptron predictors.

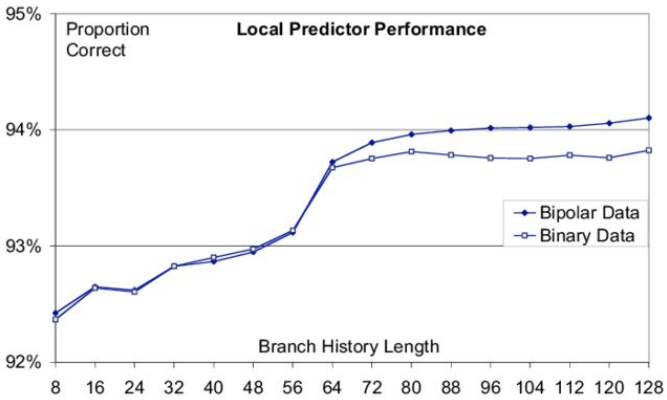

Fig. 13. Accuracy of local only perceptron predictors.

Figures 12, 13, 14, and 15 show that the combination of global/local history for perception predictors can yield more

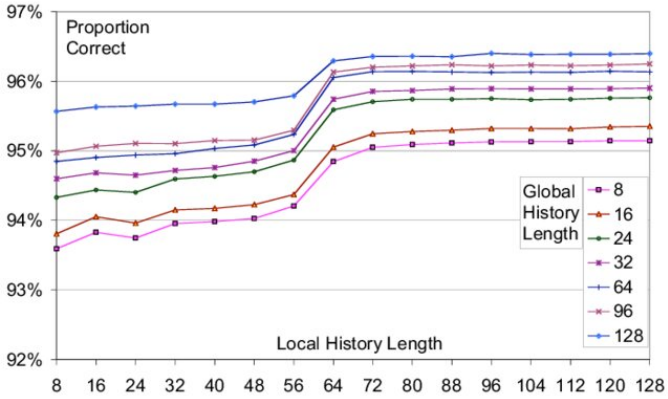

Fig. 14. Accuracy of bipolar global/local combined perception predictors.

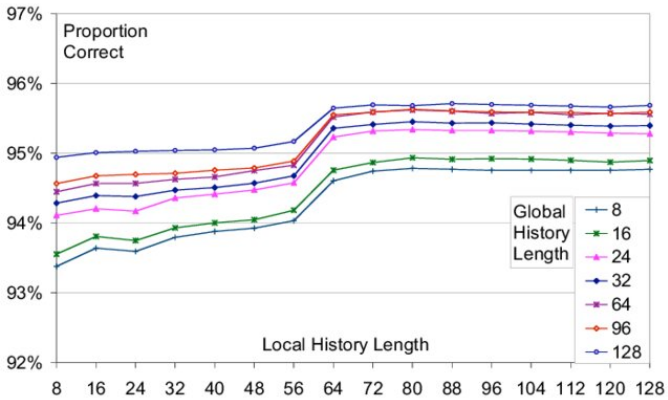

Fig. 15. Accuracy of binary global/local combined perceptron predictors.

accurate predictions in comparison to perceptron predictors that only global or local history alone.

VII. Overriding Perceptron Predictors

Another version of the predictor using perceptrons is the overriding perceptron predictor, which can address a potential problem of any complex predictor: delay. This perceptron predictor uses two levels, where a second-level perceptron predictor is combined with a first-level GShare predictor. This strategy helps increase the predictor's overall performance. This helps increase the IPC. Figure 16 shows the IPC of a 2-cycle overriding predictor with perceptrons is consistently the highest throughout every benchmark.

VIII.

IX. Conclusion

Designers of pipelined computers are striving to create the most accurate branch prediction algorithm possible because quick and accurate predictors lead to faster processors. This paper has demonstrated that perceptrons are a viable solution to improving branch predictor performance.

Perceptrons have some advantages over traditional branch predictors. Namely, perceptrons grow linearly with the size of their inputs whereas pattern history table based predictors

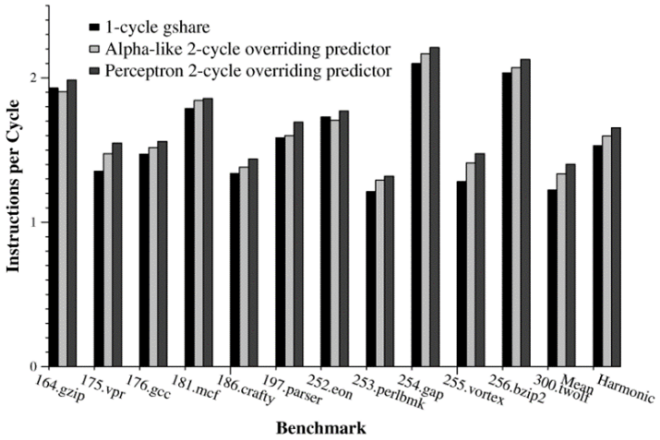

Fig. 16. The IPC for 1-cycle gshare, alpha-like 2-cycle overriding predictor, and perceptron 2-cycle overriding predictor.

grow exponentially. This trait allows perceptrons to process much more data than pattern history table based predictors.

Moreover, perceptrons treat branch prediction differently from traditional predictors in that they "learn" about branch behavior instead of following set rules like a finite state machine. This makes perceptrons more adaptive than the traditional predictors covered in this paper, as well as allowing perceptrons to improve their accuracy over time.

While perceptrons may seem like a silver bullet, they have at least one limitation: they can only separate linearly separable data. This means it is possible for specially designed branch predictors to outperform perceptrons on programs with complicated branching behavior.

Finally, we showed that perceptron-based predictors can consistently outperform traditional branch predictors. Given the right application, perceptron-based branch predictors are a better choice than traditional branch predictors.

References

[1] M. Lefkowitz, "Professor's Perceptron Paved the Way for AI – 60 Years Too Soon," *Cornell Chronicle*, 2019.

[2] ritvikmath, "Perceptron," 2019. [Online]. Available: https://www.youtube.com/watch?v=4Gac5I64LM4&t=184s. [Accessed: 2024].

[3] C. L. Daniel Jimenez, "Neural Methods for Dynamic Branch Prediction," *ACM*.

[4] Maurizio Gabbrielli, "Neural Networks," University of Bologna, 2024. [Online]. Available: http://www.cs.unibo.it/gabbri/MaterialeCorsi/Lezione17.4pPage.pdf. [Accessed: 2024].